

# rebound\_\_rust — Provenance of the Pure-Rust Port of REBOUND

## 5.1.1

rustSolveIt / Windows 11 port verification record

August 22, 2026

### Abstract

This document is the complete, self-contained provenance of the port of the REBOUND N-body code from C to pure Rust on Windows 11: how the C reference was obtained and built, how every C source file was translated, every command used to build the Rust crate, how the port was verified bit-for-bit against the MSVC-compiled C reference, and complete instructions for using the pure Rust code. **Every ported subsystem reproduces the MSVC C reference build bit-for-bit:** 63 integrator configurations, a 1482-particle shearing-sheet run with  $\sim 10^5$  collisions (identical SHA-256 of the state dump), all 65 orbital-derivative functions, the frequency analysis in all three modes, and cross-language Simulationarchive round trips in both directions.

## Identification

- Crate: rebound\_rs version 5.1.1, at C:\Users\nsh\Developer\github\rustSolveIt\_Win11\_SUNDIALS\_7\_8\_0\rebound
- Upstream: <https://github.com/hannorein/rebound>, commit dad5f97806ecbb408dcaff728851c64e67f9f6eb (“Patch (#931)”, version 5.1.1)
- License: GPL-3.0-or-later (derivative work of REBOUND, © Hanno Rein and collaborators)

## 1 Machine and toolchain

| Component        | Value                                                                                          |
|------------------|------------------------------------------------------------------------------------------------|
| OS               | Windows 11 Pro for Workstations 10.0.26200, x86-64 (no WSL2 involved anywhere)                 |
| C compiler       | MSVC c1 19.51.36256 for x64 (Visual Studio 2026 Build Tools)                                   |
| MSVC environment | "C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat" |
| make             | GnuWin32 Make 3.81 (winget install GnuWin32.Make)                                              |
| vcpkg            | C:\Users\nsh\vcpkg\vcpkg.exe                                                                   |
| Rust             | rustc 1.91.1, cargo 1.91.1, target x86_64-pc-windows-msvc                                      |

No GCC, no Clang, no WSL2 compiler was used at any point.

## 2 Step 1 — GLFW via vcpkg

```
C:\Users\nsh\vcpkg\vcpkg.exe install glfw3:x64-windows
```

This produced glfw3 **3.5.1** (import library C:\Users\nsh\vcpkg\installed\x64-windows\lib\glfw3dll1.lib). REBOUND's build system *forces* `OPENGL=0` on Windows (`src/Makefile.defs:21`), so the C reference build does not link GLFW; the browser-based visualization (SERVER mode) is used instead. The vcpkg installation is staged for any future work compiling the POSIX OpenGL path.

### 3 Step 2 — Cloning and building the C reference

```
cd C:\Users\nsh\Developer\github\rustSolveIt_Win11_SUNDIALS_7_8_0
git clone https://github.com/hannorein/rebound.git rebound\rebound
cd rebound\rebound\examples\shearing_sheet
cmd /c 'set PATH=C:\Program Files (x86)\GnuWin32\bin;%PATH% && "C:\Program Files (x86)\
Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat" && make'
```

Two Windows traps: (1) `%PATH%` in a `cmd /c` string expands at parse time, so GnuWin32 must be prepended *before* `vcvars64` runs; (2) C compilations must go through PowerShell's `cmd /c` — Git-Bash's `cmd //c` can fail silently and leave stale executables.

Every C file compiles as

```
c1 -c /DBUILDINGLIBREBOUND /D_GNU_SOURCE /D_CRT_SECURE_NO_WARNINGS /
D_CRT_NONSTDC_NO_WARNINGS /Ox /fp:precise -DSERVER -DGITHASH=
dad5f97806ecbb408dcaff728851c64e67f9f6eb /Fo:rebound.obj rebound.c
```

i.e. `/Ox /fp:precise`, `OPENGL` off, `SERVER` on, no OpenMP, no MPI, no AVX-512 (the `integrator_whfast512.s` assembly is only compiled by GCC/Clang). The resulting `librebound.lib/librebound.dll` and `rebound.exe` are the reference all Rust results are compared against. On Windows, REBOUND vendors glibc's `rand_r` LCG in `rebound.c` (three rounds of  $s \mapsto 1103515245s + 12345$ , `REB_RAND_MAX` = 2147483647), reproduced exactly in the port.

## 4 The Rust crate

29 modules, 19,075 lines; 9 runnable examples (each with a Jupyter notebook); C reference harnesses under `porttest\`. Translation rules: `#![forbid(unsafe_code)]`, `#![deny(warnings)]` — **zero unsafe**, **zero external dependencies**, **zero warnings**; C function/struct/constant names preserved exactly; control flow, constants and **arithmetic order preserved expression by expression**. The `[workspace]` stanza in `Cargo.toml` stops cargo's upward workspace discovery from picking up a stray broken `Cargo.toml` higher in the directory tree.

### 4.1 File-by-file accounting

| C file | Rust module | Status |
|--------|-------------|--------|
|--------|-------------|--------|

|                                                            |                             |                                                                                                                                                                                        |
|------------------------------------------------------------|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rebound.c                                                  | tools.rs, server.rs, lib.rs | Ported (messages, exit, strcmp_ignore_whitespace, check_fp_contract, favicon, version/githash). N/A: allocators, SIGINT, custom-integrator registry, AVX-512 detection, OpenMP setter. |
| simulation.c                                               | simulation.rs               | Ported + verified.                                                                                                                                                                     |
| particle.c                                                 | particle.rs                 | Ported + verified.                                                                                                                                                                     |
| tools.c                                                    | tools.rs                    | Ported + verified (RNG family, energies, com family, plummer, orbit conversions, add_fmt + solar-system data, MEGNO).                                                                  |
| boundary.c                                                 | boundary.rs                 | Ported + verified (incl. shear ghost boxes).                                                                                                                                           |
| tree.c                                                     | tree.rs                     | Ported + verified (octree as index arena).                                                                                                                                             |
| gravity.c                                                  | gravity.rs                  | Ported + verified (basic/compensated/tree/jacobi + variational).                                                                                                                       |
| collision.c                                                | collision.rs                | Ported + verified (all searches and resolvers).                                                                                                                                        |
| output.c                                                   | output.rs                   | Ported (screenshot excluded with display).                                                                                                                                             |
| transformations.c                                          | transformations.rs          | Ported + verified.                                                                                                                                                                     |
| rotations.c                                                | rotations.rs                | Ported + verified.                                                                                                                                                                     |
| derivatives.c                                              | derivatives.rs              | Ported + verified: all 65 functions.                                                                                                                                                   |
| frequency_analysis.c                                       | frequency_analysis.rs       | Ported + verified (MFT/FMFT/FMFT2).                                                                                                                                                    |
| integrator_none/sei/leapfrog/ias15/whfast/saba/janus/eos.c | one module each             | Ported + verified (all tested configurations bit-identical, §6).                                                                                                                       |
| integrator_mercurius.c                                     | integrator_mercurius.rs     | Ported + verified (IAS15 encounter machinery incl. hooks).                                                                                                                             |
| integrator_bs.c                                            | integrator_bs.rs            | Ported + verified (full reb_ode framework).                                                                                                                                            |
| integrator_trace.c                                         | integrator_trace.rs         | Ported + verified (reversible checks, BS/IAS15 encounter and pericenter paths).                                                                                                        |
| integrator_whfast512.c                                     | integrator_whfast512.rs     | Windows-stub parity: under c1 the C compiles only error stubs; the Rust reproduces exactly that.                                                                                       |
| binarydata.c                                               | binarydata.rs               | Ported + verified (same byte format via explicit per-field serializers).                                                                                                               |
| simulationarchive.c                                        | simulationarchive.rs        | Ported + verified (snapshot index, diff-blob append, auto-snapshot heartbeat).                                                                                                         |
| server.c                                                   | server.rs                   | Ported + verified (HTTP endpoints, base64, threading model adapted to safe Rust).                                                                                                      |
| fmemopen.c                                                 | —                           | N/A: replaced by <code>std::io::Cursor</code> .                                                                                                                                        |
| display.c, glad.c                                          | —                           | Excluded: OpenGL (not in the Windows C build either).                                                                                                                                  |
| communication_mpi.c                                        | —                           | Excluded: MPI (not in the Windows C build).                                                                                                                                            |

---

## 5 Deviation classes (mechanical only)

1. Ownership instead of pointers (Vec-based particles, octree index arena, `Option<usize>` particle names; `ap/sim` back-pointers dropped).
2. Integrator state: C's `void*` behind a vtable becomes an enum; steps take the state out (`std::mem::replace`) and put it back; MERCURIUS/TRACE stash it in `r.integrator` during encounter sub-integrations so gravity callbacks and particle hooks reach it exactly where the C reads `r->integrator.state`.
3. `r->map` aliasing expressed by moving the Vec in and out.
4. Binary format: particles serialized in the 112-byte x86-64 layout; synthetic ids replace heap pointers in the name slots, reproducing the C's pointer-equality protocol (archives are interchangeable, verified).
5. Server threading: shared snapshot/key-queue handshake serviced by the integrate loop replaces cross-thread aliasing of the simulation.
6. Varargs `add_fmt` takes an ordered argument slice.
7. frequency-analysis sort: Rust stable sort vs C `qsort` (identical permutation for distinct keys).
8. OpenMP/`reb_sigint`/MPI branches omitted (never compiled in the reference build). Uninitialized C memory is zero-initialized in Rust.

## 6 Verification

Methodology: run the identical experiment in the MSVC C reference and in Rust, dump every value as raw IEEE-754 bit patterns, compare byte-for-byte. Foundation (200,000-sample libm differential): Rust on `x86_64-pc-windows-msvc` matches the UCRT bit-for-bit for `sin`, `cos`, `tan`, `atan2`, `sqrt`, `fmod`, `exp`, `log`; **pow is the single divergent function** (60/200,000 inputs,  $\leq 2$  ulp; Rust ships its own implementation). `cbrt` was exercised bit-exactly through MERCURIUS and `add_fmt`.

All C harnesses compile from `porttest\` with:

```
cmd /c 'C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat" >nul && cl /nologo /I"....\rebound\rebound\src" /D_GNU_SOURCE /D_CRT_SECURE_NO_WARNINGS /D_CRT_NONSTDC_NO_WARNINGS /Ox /fp:precise integrators_test.c librebound.lib /Fe:integrators_test.exe'
```

### 6.1 Integrator matrix

Three-body problem (star  $m = 1$ ; planet  $m = 10^{-3}$ ,  $x = 1.6$ ,  $v_y = 0.5$ ; moon  $m = 10^{-7}$ ,  $x = 1.7$ ,  $v_y = 0.6$ ,  $z = 0.01$ ,  $v_z = 0.001$ ),  $G = 1$ ,  $dt = 0.01$ , 500 steps. **Result: 63 of 63 configurations bit-identical:**

- none; ias15 (plus a 1000-step adaptive run); leapfrog 2/4/6/8;
- whfast: default, c11, c17+corrector2, dh, whds, bary, mk, comp, lazy, usafe;
- saba  $\times 13$  (incl. CM/CL variants); janus 2–10; eos  $\times 13$ ;
- mercurius  $\times 6$  — the default keeps the moon inside the planet's critical radius so the IAS15 encounter machinery runs every step (5000-step run additionally verified);

- `bs × 4` (adaptive, incl. the `powf`-driven step control);
- `trace × 6` — cross-checked to genuinely take the BS-encounter and pericenter FULL paths.

## 6.2 Shearing sheet

Stock Saturn’s-rings example (SEI, octree gravity, tree collisions, shear boundary, Bridges restitution, `rand_r` ICs), seed 42, 1482 particles, 400 steps,  $\sim 10^5$  collisions:

```
.\rebound_test.exe 400
..\target\release\examples\shearing_sheet_test.exe 400
SHA-256(state_c_final.txt) = 75
    BDAAB7109F125192F56AEB0CCDCAC554AF88A165FE11075EC5A871178521F0
SHA-256(state_rust_final.txt) = 75
    BDAAB7109F125192F56AEB0CCDCAC554AF88A165FE11075EC5A871178521F0
```

**Byte-identical** initial and final dumps. (The Bridges law is written via `exp/log` identically on both sides to keep the documented `pow` difference out of the harness; the full bisection history — 330 drifted particles, first divergence at step 78, isolated to collision pair (390, 1456), traced to `pow` — is in `shearing_sheet_port_test.md`, which repeats all commands.)

## 6.3 Derivatives, frequency analysis, `add_fmt`

All 65 `reb_particle_derivative_*` functions over two configurations: 130/130 lines bit-identical. Frequency analysis (256-sample three-frequency signal): MFT, FMFT, FMFT2 all bit-identical. `add_fmt` (solar-system dataset, orbital elements, Pal coordinates, period path): all 12 particles bit-identical.

## 6.4 Simulationarchive cross-language round trips

Each implementation runs  $3 \times 100$  steps saving snapshots; the *other* implementation loads the 200-step snapshot and continues 100 steps, landing on the writer’s 300-step state **bit-exactly** — in both directions, for `whfast-usafe` (unsynchronized Jacobi coordinates) and `ias15` (adaptive restart arrays). The binary format including the incremental diff-blob append is fully compatible between C and Rust.

## 6.5 Web server

The Rust server paused a 100-step simulation; the blob fetched from `http://localhost:12873/simulation` was loaded by the C build to the bit-identical state; `/keyboard/81` cleanly terminated the paused run.

# 7 Building the crate — all cargo commands

```
cd C:\Users\nsh\Developer\github\rustSolveIt_Win11_SUNDIALS_7_8_0\rebound_rust
cargo build --release
cargo build --release --example shearing_sheet
cargo build --release --example shearing_sheet_test
cargo build --release --example integrators_test
cargo build --release --example libm_diff
cargo build --release --example derivatives_test
cargo build --release --example frequency_test
cargo build --release --example archive_test
cargo build --release --example server_test
```

```
cargo build --release --example addfmt_test
```

Zero warnings under `forbid(unsafe_code) + deny(warnings)`; std-only, builds offline. No fast-math or FMA contraction in either language (`reb_check_fp_contract()` returns 0).

## 8 Known limitations

(1) pow:  $\leq 2$  ulp divergence on  $\sim 0.03\%$  of inputs (affects `reb_random_powerlaw` and user code; all `powf` calls observed in the BS controller matched bitwise). (2) WHFast512 integrates nothing on Windows — in C and Rust alike. (3) OpenGL display and MPI excluded. (4) The C's documented integrator restrictions carry over unchanged.

## 9 Using the pure Rust code

Add to `Cargo.toml`:

```
[dependencies]
rebound_rs = { path = "C:/Users/nsh/Developer/github/rustSolveIt_Win11_SUNDIALS_7_8_0/
  rebound_rust" }
```

Minimal example (names are the C names):

```
use rebound_rs::*;

fn main() {
    let mut sim = reb_simulation_create();
    let r = &mut sim;
    reb_simulation_set_integrator(r, "whfast");
    r.G = 1.0;
    r.dt = 0.01;

    let mut star = reb_particle::default();
    star.m = 1.0;
    reb_simulation_add(r, star);

    let mut planet = reb_particle::default();
    planet.m = 1e-3; planet.x = 1.6; planet.vy = 0.5;
    reb_simulation_add(r, planet);

    reb_simulation_add_fmt(r, "m a e", &[
        reb_fmt_arg::d(1e-7), reb_fmt_arg::d(2.5), reb_fmt_arg::d(0.1),
    ]);

    reb_simulation_move_to_com(r);
    reb_simulation_integrate(r, 100.0);
    println!("t = {} E = {}", r.t, reb_simulation_energy(r));
}
```

Integrator options are set on the state enum:

```
if let reb_integrator_state::whfast(ref mut wh) = r.integrator {
    wh.corrector = 17;
```

```
    wh.safe_mode = 0;
}
```

Simulation archives (interchangeable with C-REBOUND files):

```
reb_simulation_save_to_file(r, Some("archive.bin"));
reb_simulation_save_to_file_interval(r, "auto.bin", 10.0);
let mut restored = reb_simulation_create_from_file("archive.bin", -1).unwrap();
```

Browser visualization:

```
reb_simulation_start_server(r, 1234);    // open http://localhost:1234
reb_simulation_integrate(r, 1e6);
reb_simulation_stop_server(r);
```

Examples run with `cargo run -release -example <name> [- args]`; each example has a companion Jupyter notebook under `notebooks\` that builds it, runs it, and displays the output.

## 10 Licensing

REBOUND is © Hanno Rein, Shangfei Liu, Dave O’Hallaron, Ernst Hairer, Tiger Lu, Dan Tamayo, Rishit Dagli, Pejvak Javaheri and contributors, GPL-3.0-or-later. This Rust translation is a derivative work under the same license; every module header cites the C file it translates. Upstream commit: `dad5f97806ecbb408dcaff728851c64e67f9f6eb` (“5.1.1”).